

Line-by-Line Mathematical Guide for Sig_ROM.ec

Generated HAETAE EasyCrypt proof guide

2026-05-11

File Role

Generic signature games and random-oracle interfaces.

How To Read This Script

Read this file as the reusable game framework. Its scripts define abstract experiments and prove the exact NMA-to-CMA adapter fact used by the HAETAE-specific proof.

Line-by-Line Mathematical Reading

Each row preserves one EasyCrypt source line and gives the intended mathematical or proof-script reading. Blank rows are kept because they mark proof structure in long EasyCrypt developments.

Line	EasyCrypt source	Mathematical reading
1	<code>require import AllCore List PROM.</code>	Imports AllCore List PROM. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
2	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3	<code>abstract theory Sig_ROM.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
5	<code>type pkey.</code>	Declares carrier set pkey. EasyCrypt will type-check all later functions and games over this mathematical domain.
6	<code>type skey.</code>	Declares carrier set skey. EasyCrypt will type-check all later functions and games over this mathematical domain.
7	<code>type message.</code>	Declares carrier set message. EasyCrypt will type-check all later functions and games over this mathematical domain.
8	<code>type context.</code>	Declares carrier set context. EasyCrypt will type-check all later functions and games over this mathematical domain.
9	<code>type signature.</code>	Declares carrier set signature. EasyCrypt will type-check all later functions and games over this mathematical domain.
10	<code>type ro_query.</code>	Declares carrier set ro_query. EasyCrypt will type-check all later functions and games over this mathematical domain.
11	<code>type ro_output.</code>	Declares carrier set ro_output. EasyCrypt will type-check all later functions and games over this mathematical domain.
12	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
13	<code>op [lossless] dro_output : ro_query -> ro_output distr.</code>	Defines dro_output as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.

Line	EasyCrypt source	Mathematical reading
14	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
15	<code>type query = message * context.</code>	Declares carrier set <code>query</code> . EasyCrypt will type-check all later functions and games over this mathematical domain.
16	<code>type signed_query = query * signature.</code>	Declares carrier set <code>signed_query</code> . EasyCrypt will type-check all later functions and games over this mathematical domain.
17	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
18	<code>op fresh_msg (qs : query list) (m : message) (ctx : context) : bool =</code>	Defines predicate <code>fresh_msg</code> , an event or well-formedness condition used in later implications and game proofs.
19	<code>! ((m, ctx) \in qs).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
20	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
21	<code>op fresh_sig (qs : signed_query list) (m : message) (ctx : context)</code>	Defines operation <code>fresh_sig</code> , a total mathematical function or abbreviation used by later declarations.
22	<code>(sig : signature) : bool =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
23	<code>! (((m, ctx), sig) \in qs).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
24	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
25	<code>lemma fresh_msg_nil m ctx : fresh_msg [] m ctx.</code>	States checked lemma <code>fresh_msg_nil</code> . The following proof script must derive this proposition from prior definitions and lemmas.
26	<code>proof. by rewrite /fresh_msg. qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
27	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
28	<code>lemma fresh_sig_nil m ctx sig : fresh_sig [] m ctx sig.</code>	States checked lemma <code>fresh_sig_nil</code> . The following proof script must derive this proposition from prior definitions and lemmas.
29	<code>proof. by rewrite /fresh_sig. qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
30	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
31	<code>clone import FullRO as RO with</code>	Clones and opens a parameterized EasyCrypt theory, producing a specialized instance used below.
32	<code>type in_t <- ro_query,</code>	Declares carrier set <code>in_t</code> . EasyCrypt will type-check all later functions and games over this mathematical domain.
33	<code>type out_t <- ro_output,</code>	Declares carrier set <code>out_t</code> . EasyCrypt will type-check all later functions and games over this mathematical domain.
34	<code>op dout <- dro_output.</code>	Defines operation <code>dout</code> , a total mathematical function or abbreviation used by later declarations.
35	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
36	<code>module type Oracle = {</code>	Defines module interface <code>Oracle</code> . It specifies the procedures that later games or adversaries may call.
37	<code>include FRO [init, get]</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
38	<code>}. <i>blank</i></code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
39	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
40	<code>module type POracle = {</code>	Defines module interface <code>POracle</code> . It specifies the procedures that later games or adversaries may call.
41	<code>include FRO [get]</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
42	<code>}. <i>blank</i></code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
43	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
44	<code>module type Scheme(O : POracle) = {</code>	Defines module interface <code>Scheme</code> . It specifies the procedures that later games or adversaries may call.
45	<code>proc kg() : pkey * skey</code>	Defines procedure <code>kg</code> . Its body is a probabilistic program whose result becomes part of a game event.
46	<code>proc sign(sk : skey, m : message, ctx : context) : signature</code>	Defines procedure <code>sign</code> . Its body is a probabilistic program whose result becomes part of a game event.

Line	EasyCrypt source	Mathematical reading
47	<code>proc verify(pk : pkey, m : message, ctx : context,</code>	Defines procedure verify. Its body is a probabilistic program whose result becomes part of a game event.
48	<code>sig : signature) : bool</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
49	<code>}.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
50	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
51	<code>module type CORR_ADV(O : POracle) = {</code>	Defines module interface CORR_ADV. It specifies the procedures that later games or adversaries may call.
52	<code>proc find(pk : pkey, sk : skey) : message * context</code>	Defines procedure find. Its body is a probabilistic program whose result becomes part of a game event.
53	<code>}..</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
54	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
55	<code>module Correctness(H : Oracle, S : Scheme, A : CORR_ADV) = {</code>	Defines module Correctness, a probabilistic program or game used to create events and success probabilities.
56	<code>proc main() : bool = {</code>	Defines procedure main. Its body is a probabilistic program whose result becomes part of a game event.
57	<code>var pk : pkey;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
58	<code>var sk : skey;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
59	<code>var m : message;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
60	<code>var ctx : context;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
61	<code>var sig : signature;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
62	<code>var ok : bool;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
63	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
64	<code>H.init();</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
65	<code>(pk, sk) <@ S(H).kg();</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
66	<code>(m, ctx) <@ A(H).find(pk, sk);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
67	<code>sig <@ S(H).sign(sk, m, ctx);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
68	<code>ok <@ S(H).verify(pk, m, ctx, sig);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
69	<code>return !ok;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
70	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
71	<code>}..</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
72	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
73	<code>module type SignOracle = {</code>	Defines module interface SignOracle. It specifies the procedures that later games or adversaries may call.
74	<code>proc sign(m : message, ctx : context) : signature</code>	Defines procedure sign. Its body is a probabilistic program whose result becomes part of a game event.
75	<code>}..</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
76	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
77	<code>module type Adversary(H : POracle, O : SignOracle) = {</code>	Defines module interface Adversary. It specifies the procedures that later games or adversaries may call.
78	<code>proc forge(pk : pkey) : message * context * signature {H.get, O.sign}</code>	Defines procedure forge. Its body is a probabilistic program whose result becomes part of a game event.
79	<code>}..</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
80	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
81	module type NMA_Adversary(H : POracle) = {	Defines module interface NMA_Adversary. It specifies the procedures that later games or adversaries may call.
82	proc forge(pk : pkey) : message * context * signature {H.get}	Defines procedure forge. Its body is a probabilistic program whose result becomes part of a game event.
83	}.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
84	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
85	module NMA_As_CMA(A : NMA_Adversary) (H : POracle, O : SignOracle) = {	Defines module NMA_As_CMA, a probabilistic program or game used to create events and success probabilities.
86	proc forge(pk : pkey) : message * context * signature = {	Defines procedure forge. Its body is a probabilistic program whose result becomes part of a game event.
87	var r : message * context * signature;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
88	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
89	r <@ A(H).forge(pk);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
90	return r;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
91	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
92	}.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
93	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
94	module EUF_CMA(H : Oracle, S : Scheme, A : Adversary) = {	Defines module EUF_CMA, a probabilistic program or game used to create events and success probabilities.
95	var sk : skey	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
96	var queries : query list	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
97	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
98	module O = {	Defines module O, a probabilistic program or game used to create events and success probabilities.
99	proc sign(m : message, ctx : context) : signature = {	Defines procedure sign. Its body is a probabilistic program whose result becomes part of a game event.
100	var sig : signature;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
101	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
102	sig <@ S(H).sign(sk, m, ctx);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
103	queries <- (m, ctx) :: queries;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
104	return sig;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
105	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
106	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
107	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
108	module A = A(H, O)	Defines module A, a probabilistic program or game used to create events and success probabilities.
109	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
110	proc main() : bool = {	Defines procedure main. Its body is a probabilistic program whose result becomes part of a game event.
111	var pk : pkey;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
112	var m : message;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
113	var ctx : context;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
114	var sig : signature;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
115	var ok : bool;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
116	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
117	H.init();	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
118	queries <- [];	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
119	(pk, sk) <@ S(H).kg();	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
120	(m, ctx, sig) <@ A.forge(pk);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
121	ok <@ S(H).verify(pk, m, ctx, sig);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
122	return ok /\ fresh_msg queries m ctx;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
123	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
124	}.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
125	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
126	module UF_NMA(H : Oracle, S : Scheme, A : NMA_Adversary) = {	Defines module UF_NMA, a probabilistic program or game used to create events and success probabilities.
127	proc main() : bool = {	Defines procedure main. Its body is a probabilistic program whose result becomes part of a game event.
128	var pk : pkey;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
129	var sk : skey;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
130	var m : message;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
131	var ctx : context;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
132	var sig : signature;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
133	var ok : bool;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
134	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
135	H.init();	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
136	(pk, sk) <@ S(H).kg();	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
137	(m, ctx, sig) <@ A(H).forge(pk);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
138	ok <@ S(H).verify(pk, m, ctx, sig);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
139	return ok;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
140	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
141	}..	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
142	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
143	section NMA_Embedding.	Starts section NMA_Embedding, a scoped region for related declarations and hypotheses.
144	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
145	declare module H <: Oracle {-EUF_CMA}.	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.

Line	EasyCrypt source	Mathematical reading
146	<code>declare module S <: Scheme {-H, -EUF_CMA}.</code>	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
147	<code>declare module A <: NMA_Adversary {-H, -S, -EUF_CMA}.</code>	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
148	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
149	<code>lemma nma_as_cma_exact &m :</code>	States checked lemma <code>nma_as_cma_exact</code> . The following proof script must derive this proposition from prior definitions and lemmas.
150	<code>Pr[EUF_CMA(H, S, NMA_As_CMA(A)).main() @ &m : res] =</code>	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
151	<code>Pr[UF_NMA(H, S, A).main() @ &m : res].</code>	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
152	<code>proof.</code>	Begins the proof script for the preceding proposition.
153	<code>byequiv (: = {glob H, glob S, glob A} ==> = {res}) => //.</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
154	<code>proc.</code>	Enters procedure-body proof mode for a module procedure.
155	<code>inline EUF_CMA(H, S, NMA_As_CMA(A)).A.forge</code>	Unfolds a procedure body so the proof can reason about its concrete commands.
156	<code>NMA_As_CMA(A, H, EUF_CMA(H, S, NMA_As_CMA(A)).O).forge.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
157	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
158	<code>call (: = {glob H, glob S, glob A, arg} ==> = {glob H, glob S, glob A, res}).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
159	<code>+ by sim.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
160	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
161	<code>call (: = {glob H, glob S, glob A, arg} ==> = {glob H, glob S, glob A, res}).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
162	<code>+ by sim.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
163	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
164	<code>call (: = {glob H, glob S} ==> = {glob H, glob S, res}).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
165	<code>+ by sim.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
166	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
167	<code>call (: = {glob H} ==> = {glob H}).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
168	<code>+ by sim.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
169	<code>by auto => />.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
170	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
171	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
172	<code>end section NMA_Embedding.</code>	Closes the current section, discharging local module and variable scope.
173	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
174	<code>module SUF_CMA(H : Oracle, S : Scheme, A : Adversary) = {</code>	Defines module <code>SUF_CMA</code> , a probabilistic program or game used to create events and success probabilities.
175	<code>var sk : skey</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
176	<code>var queries : signed_query list</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
177	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
178	<code>module O = {</code>	Defines module <code>O</code> , a probabilistic program or game used to create events and success probabilities.
179	<code>proc sign(m : message, ctx : context) : signature = {</code>	Defines procedure <code>sign</code> . Its body is a probabilistic program whose result becomes part of a game event.
180	<code>var sig : signature;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
181	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
182	<code>sig <@ S(H).sign(sk, m, ctx);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
183	<code>queries <- ((m, ctx), sig) :: queries;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
184	<code>return sig;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
185	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
186	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
187	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
188	<code>module A = A(H, 0)</code>	Defines module A, a probabilistic program or game used to create events and success probabilities.
189	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
190	<code>proc main() : bool = {</code>	Defines procedure main. Its body is a probabilistic program whose result becomes part of a game event.
191	<code>var pk : pkey;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
192	<code>var m : message;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
193	<code>var ctx : context;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
194	<code>var sig : signature;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
195	<code>var ok : bool;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
196	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
197	<code>H.init();</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
198	<code>queries <- [];</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
199	<code>(pk, sk) <@ S(H).kg();</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
200	<code>(m, ctx, sig) <@ A.forge(pk);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
201	<code>ok <@ S(H).verify(pk, m, ctx, sig);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
202	<code>return ok /\ fresh_sig queries m ctx sig;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
203	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
204	<code>}.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
205	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
206	<code>end Sig_ROM.</code>	Closes the current theory or module block.